Final Project

Solar System

Yu-Xian Lai
laiyux@oregonstate.edu

Text from my proposal:

I will meet all the requirements of the Solar System part in the Final Project Proposal Comments.

What I did for my project:

I studied the slide "Setting the Eye Position on an Orbiting Body" from the lecture, the physical parameter setup and the timing setup gave me a good example of how to set up the parameter of all the planets.

Based on the Earth's orbital radius, orbit time and spin time, I set up all the other planet's parameters.
For example, I set the parameters of Mars and Jupiter like this:

```
const float MARS_RADIUS_MILES = EARTH_RADIUS_MILES * 0.53;
const float MARS_TILT_ANGLE_DEG = 25.2f;
const float MARS_ORBITAL_RADIUS_MILES = EORS * 92900000.f * 1.5237 + 2;
const float MARS_ORBIT_TIME_DAYS = TS * 687.f;
const float MARS_ORBIT_TIME_HOURS = MARS_ORBIT_TIME_DAYS * 24.f;
const float MARS_ORBIT_TIME_SECONDS = MARS_ORBIT_TIME_HOURS * 60.f * 60.f;
const float MARS_SPIN_TIME_DAYS = TS * 1.0178f;
const float MARS_SPIN_TIME_HOURS = MARS_SPIN_TIME_DAYS * 24.f;
const float MARS_SPIN_TIME_SECONDS = MARS_SPIN_TIME_HOURS * 60.f * 60.f;
```

```
const float JUPITER_TILT_ANGLE_DEG = 3.1f;
const float JUPITER_ORBITAL_RADIUS_MILES = EORS * 92900000.f * 5.2028 + 2;
const float JUPITER_ORBIT_TIME_DAYS = TS * 365.3f * 11.9;
const float JUPITER_ORBIT_TIME_HOURS = JUPITER_ORBIT_TIME_DAYS * 24.f;
const float JUPITER_ORBIT_TIME_SECONDS = JUPITER_ORBIT_TIME_HOURS * 60.f * 60.f;
const float JUPITER_SPIN_TIME_DAYS = TS * 0.4154f;
const float JUPITER_SPIN_TIME_HOURS = JUPITER_SPIN_TIME_DAYS * 24.f;
const float JUPITER_SPIN_TIME_SECONDS = JUPITER_SPIN_TIME_HOURS * 60.f * 60.f;
```

The orbital radius of all planets has to add 2 which is the radius of the Sun, so that Mercury, Venus and Earth won't be blocked by the Sun.

I have followed Kepler's Third law for all planets.
The orbital radius which is the distance from the planets to sun is the same with how many astronomical units there is between the planets and the sun.

The orbital radius (a) of Earth is EORS * 92900000 which is equal to 1AU, the orbit period (p) of Earth is TS * 365 days which is scaled from 1 year.
p^2/a^3 of Earth equals to 1.

The orbital radius (a) of Mercury is EARTH_ORBIT_RADIUS * 0.3871 which is 0.3871AU, the orbit period (p) of Mercury is TS * 88 days which is 0.241 year.
p^2/a^3 of Mercury is 1.002.

The orbital radius (a) of Venus is EARTH_ORBIT_RADIUS * 0.7233 which is 0.7233AU, the orbit period (p) of Venu is TS * 225 days which is 0.615 year.
p^2/a^3 of Venus is 1.002.

The orbital radius (a) of Mars is EARTH_ORBIT_RADIUS * 1.5237 which is 1.5237AU, the orbit period (p) of Mars is TS * 687 days which is 1.881 year.
p^2/a^3 of Mars is 1.000.

The orbital radius (a) of Jupiter is EARTH_ORBIT_RADIUS * 5.2028 which is 5.2028AU, the orbit period (p) of Jupiter is TS * 365 * 11.9 days which is 11.9 year.
p^2/a^3 of Jupiter is 0.999.

The orbital radius (a) of Saturn is EARTH_ORBIT_RADIUS * 9.5388 which is 9.5388AU, the orbit period (p) of Saturn is TS * 365 * 29.5 days which is 29.5 year.
p^2/a^3 of Saturn is 1.000.

The orbital radius (a) of Uranus is EARTH_ORBIT_RADIUS * 19.1914 which is 19.1914AU, the orbit period (p) of Uranus is TS * 365 * 84 days which is 84 year.
p^2/a^3 of Uranus is 0.999.

The orbital radius (a) of Neptune is EARTH_ORBIT_RADIUS * 30.0611 which is 30.0611AU, the orbit period (p) of Uranus is TS * 365 * 164.8 days which is 164.8 year.
p^2/a^3 of Uranus is 1.000.

All the planet's Orbital period is proportional to $\text{OrbitalRadius}^{3/2}$

Then, I used the rotate and translate of glm to keep the planets tilt in the same direction and then spin by itself and translate by its orbit.

```
glm::mat4
MakeMarsMatrix()
{
    const float marsSpinAngle = ONE_FULL_TURN * Time * TimeScale / MARS_SPIN_TIME_SECONDS;
    const float marsOrbitAngle = ONE_FULL_TURN * Time * TimeScale / MARS_ORBIT_TIME_SECONDS;
    const glm::mat4 identity = glm::mat4(1.);
    const glm::vec3 yaxis = glm::vec3(0., 1., 0.);
    const glm::vec3 zaxis = glm::vec3(0., 0., 1.);
    glm::vec3 morbitPosition = glm::vec3(MARS_ORBITAL_RADIUS_MILES * cosf(marsOrbitAngle), 0., -MARS_ORBITAL_RADIUS_MILES * sinf(marsOrbitAngle));
    glm::mat4 mtransx = glm::translate(identity, morbitPosition);
    glm::mat4 mtilt = glm::rotate(identity, glm::radians(MARS_TILT_ANGLE_DEG), zaxis);
    glm::mat4 mrspiny = glm::rotate(identity, marsSpinAngle - marsOrbitAngle, yaxis);
    return mtransx * mtilt * mrspiny;// 3 * 2 * 1
}
```

The TimeScale is a variable that can control how fast the planets orbit, TimeScale add 10 when pressing key “+”, minus 10 when pressing key ”-”, so I can make sure that the outer planets do not move too slow and the inner planets do not move too fast.

In display() ,I do point – light around the sun and add some ambient in there, the other planets have lit textures using GL_MODULATE, I draw the orbit circle for the planets and call the planets out.

How your project differs from what you proposed, and why:

I have meet all the requirements of what I need to do with the solar system, I’ve also added the Saturn Ring and play with eye positions on orbital body.

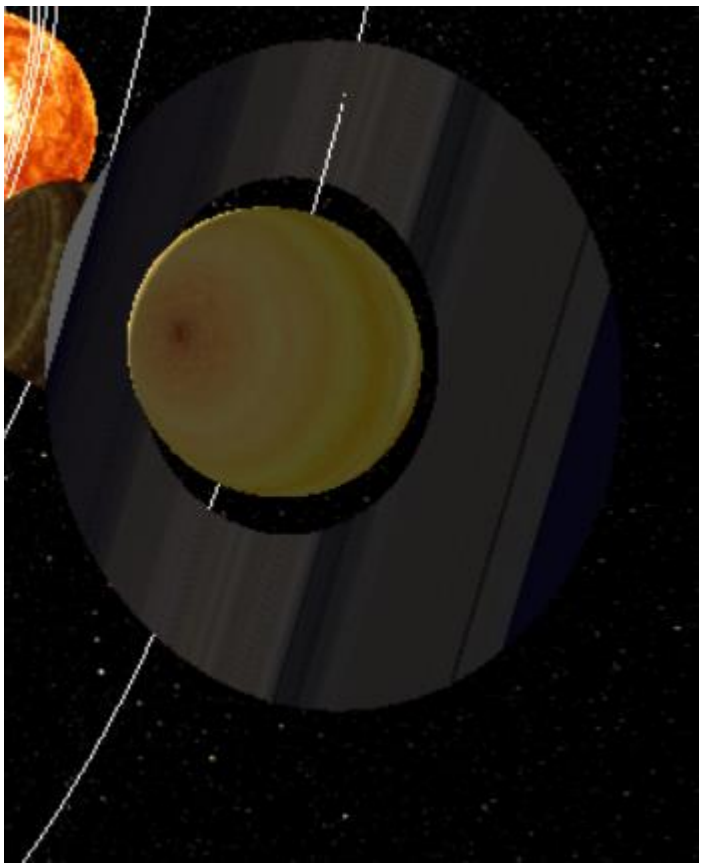
What you learned from doing this project:

I have learned that the tilt of the planet will mess up the gltranslate and glrotate, it is going to change as the planet rotate, but the planet should stay at the tilt all the way through its rotation. Therefore, we use glm to pick up the planet and put it where it belongs. I’ve got more familiar with using glm in this project.

I also tried to add the Saturn Ring, I used gluDisk to set up the Saturn Ring which the inner radius is 46298 miles, and the outer radius is 84991 miles.

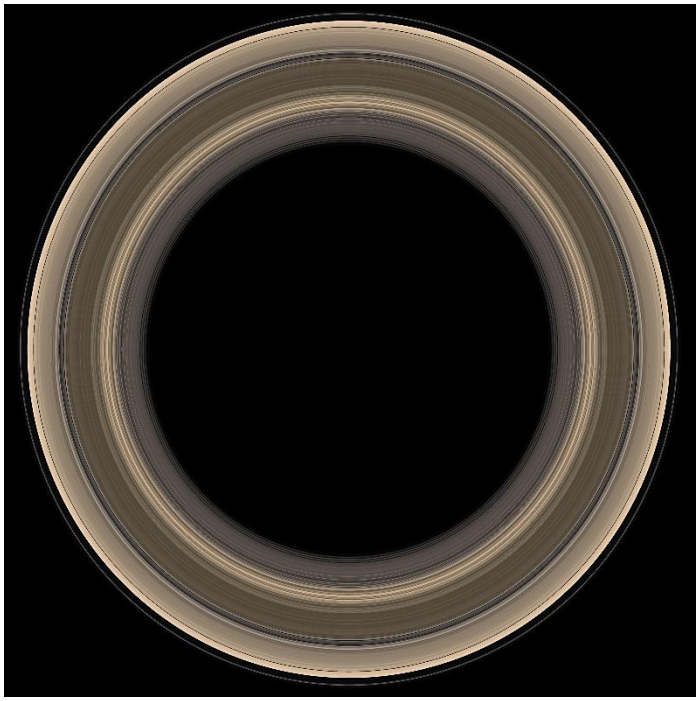
Then, I tried to texture the ring by using the Saturn Ring texture form NASA:

However, opengl would just map it to the ring based on the s and t. We got this:

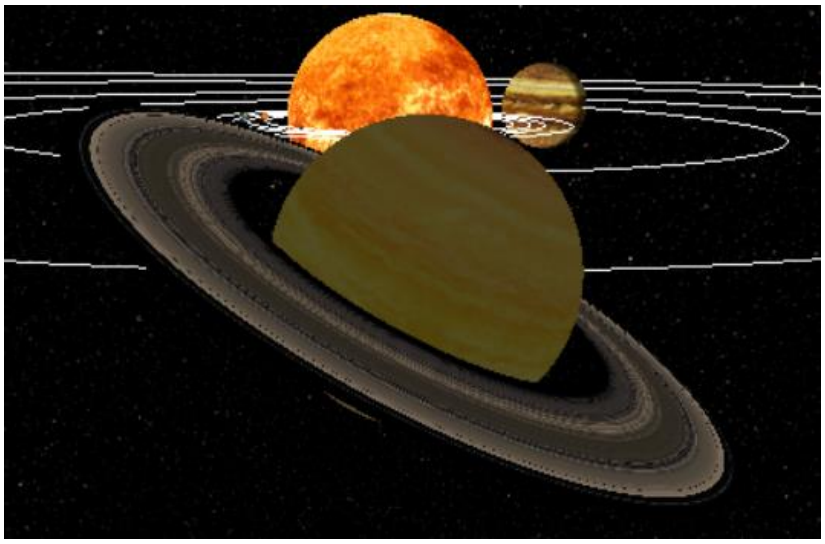


Which is not what we want.

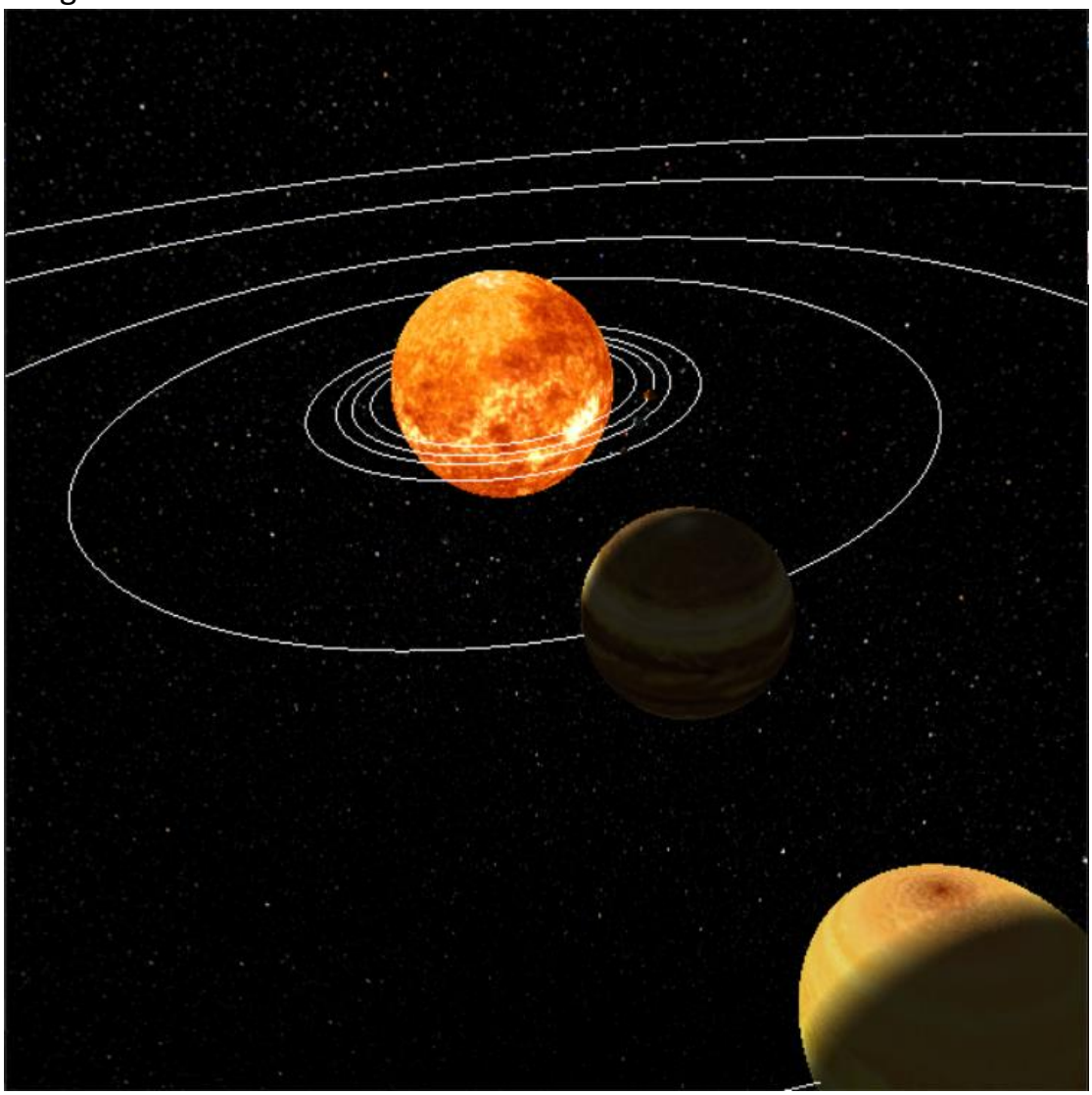
The methods I found to solve this problem is to either use the U / Follow Active Quads unwrap option on blender or make I ring of ‘particles’ using instancing. Either way can’t be finished in a week, so I tried to find a better bmp image that could be used on the texture mapping and I found the Saturn ring created by Niko22966 which could be map to the ring perfectly.

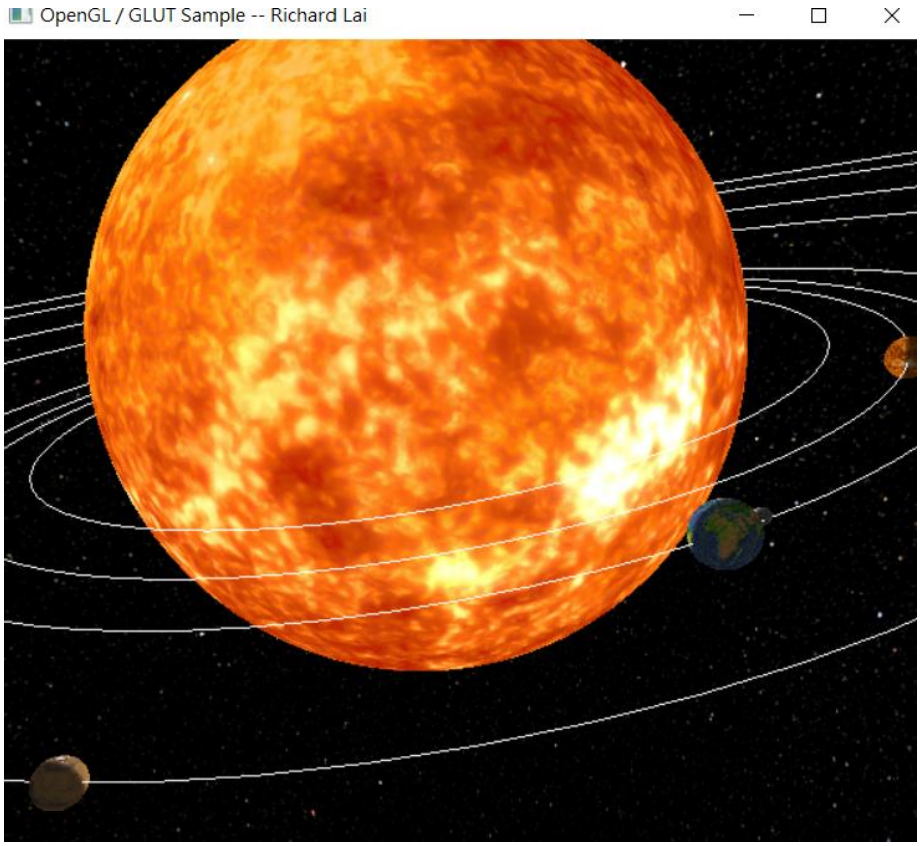


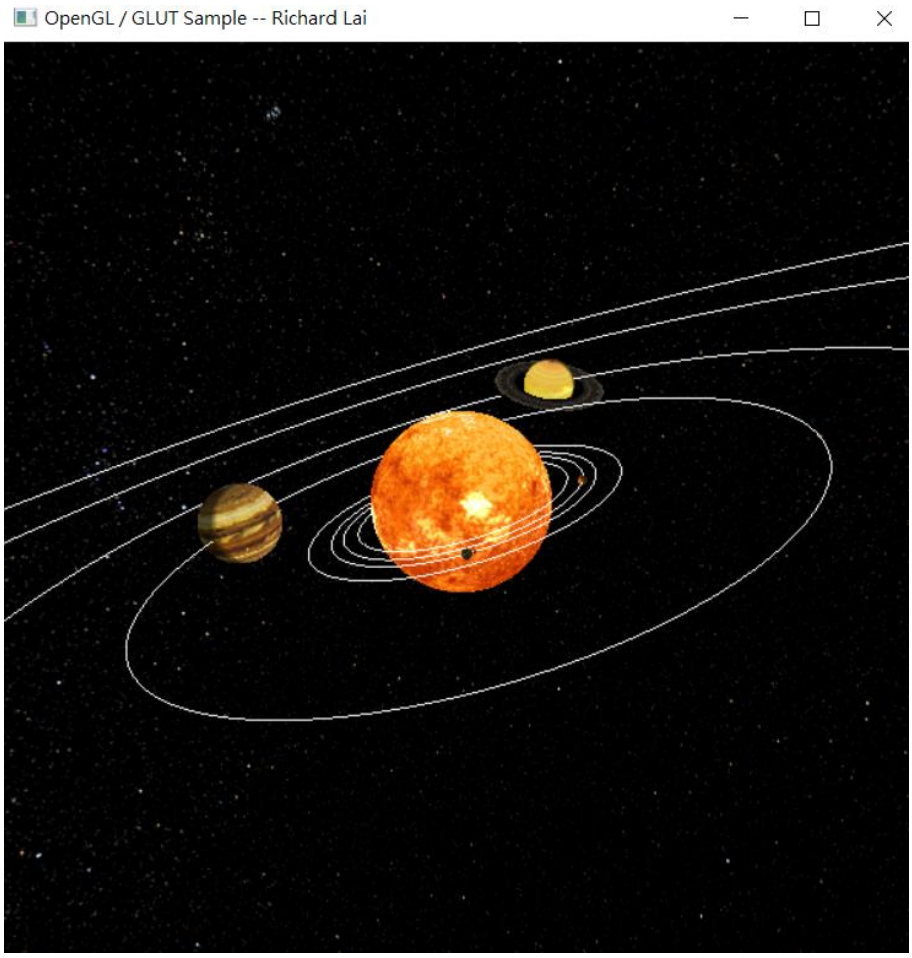
After texture mapping:

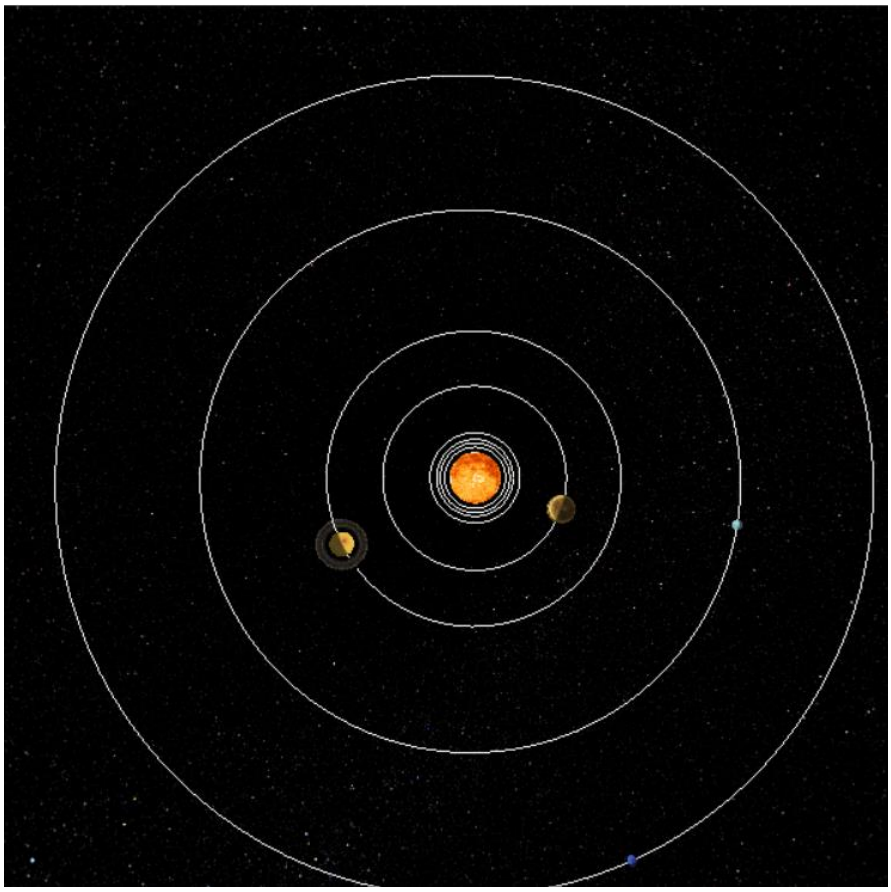


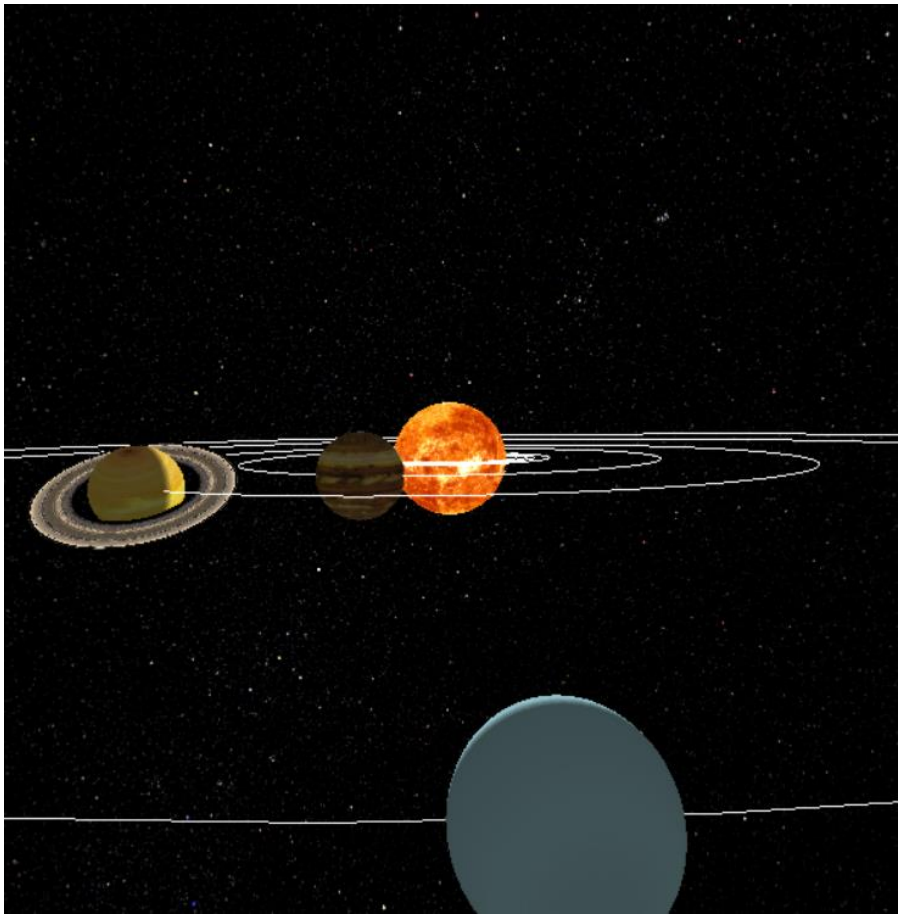
Images:

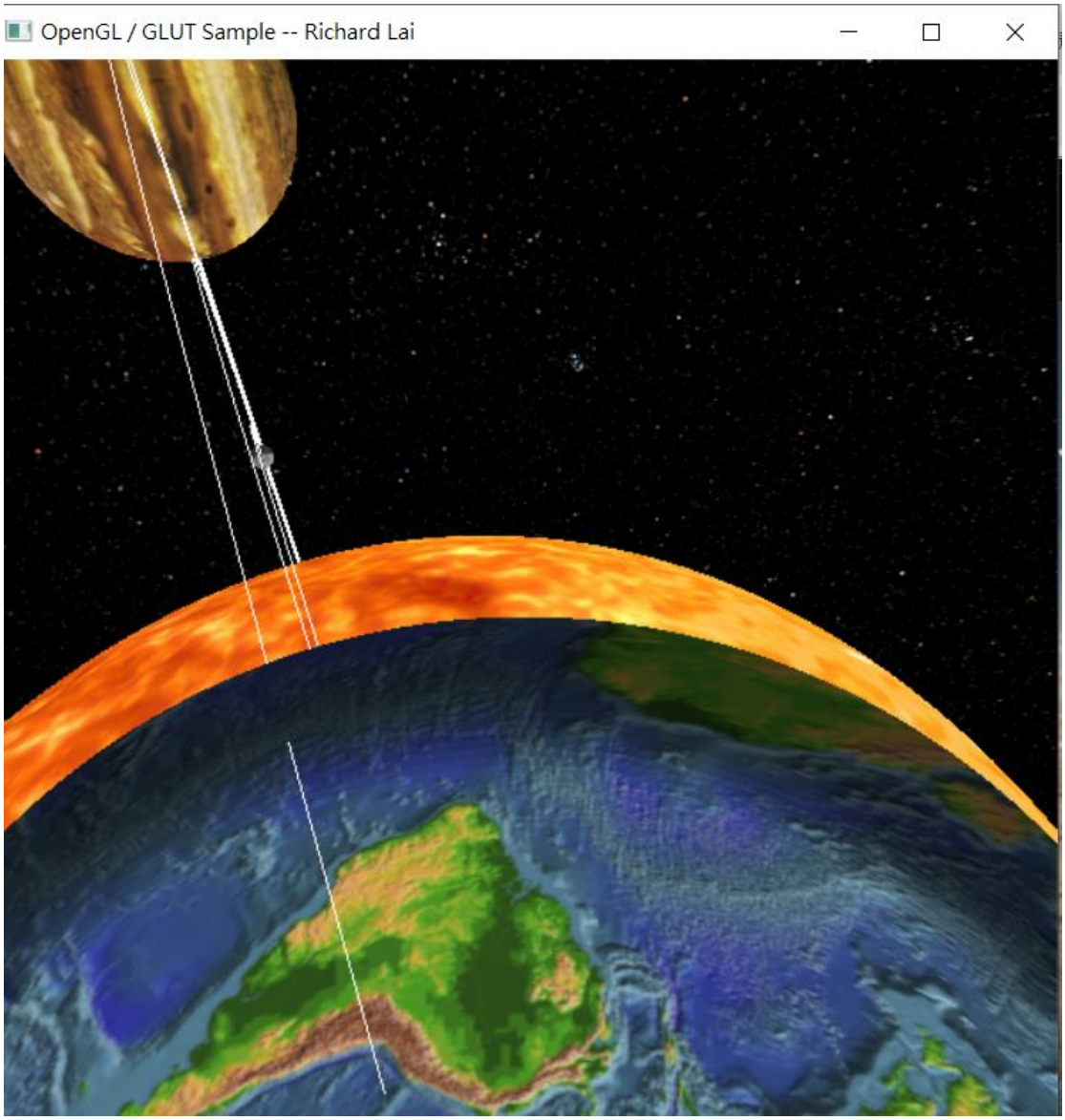



OpenGL / GLUT Sample -- Richard Lai


OpenGL / GLUT Sample -- Richard Lai







Link: https://media.oregonstate.edu/media/1_sz3lxfc3